

Documentatie tehnica — Sistem de Rezervari Cinema

Tema 3121B — Programare Orientata pe Obiecte (C++)

Grupa 3121B · An universitar 2025-2026

Document insotitor pentru codul sursa al proiectului.

Cuprins

1. Descrierea generala a sistemului
2. Diagrama UML — clase si relatii
3. Descrierea claselor
4. Concepte POO aplicate
5. Interfata web si serverul HTTP C++
6. API REST — endpoint-uri
7. Fluxul unei rezervari
8. Formula de calcul a pretului
9. Ierarhia exceptiilor
10. Strategie de testare
11. Instructiuni de build si rulare
12. Posibile imbunatatiri

1. Descrierea generala a sistemului

Proiectul implementeaza un **sistem complet de rezervari pentru un cinema**, cu doua interfete independente:

- **Aplicatie consola** (Linux/WSL) - meniu interactiv text, autentificare staff, rezervari simple si consecutive, rapoarte, persistenta in fisiere.
- **Interfata web** - server HTTP scris integral in C++ (fara librarii externe), servita la `http://localhost:8080`, cu harta interactiva a salii, login staff, panou de administrare si mod rezervare consecutiva.

Ambele interfete folosesc **acelasi nucleu C++** (clasele de domeniu, logica de pret, exceptiile), garantand consistenta datelor.

Structura repository

```
cinema-poo/  
+-src/                # Cod sursa C++ (clase de domeniu + logica)  
|  +-Film.h / Film.cpp  
|  +-Sala.h / Sala.cpp
```

```

| +-SalaVIP.h / SalaVIP.cpp
| +-Spectacol.h / Spectacol.cpp
| +-Rezervare.h / Rezervare.cpp
| +-RezervareOnline.h / RezervareOnline.cpp
| +-Persoana.h / Persoana.cpp
| +-Angajat.h / Angajat.cpp
| +-Cinematograf.h / Cinematograf.cpp
| +-SistemAutentificare.h / SistemAutentificare.cpp
| +-Persistenta.h / Persistenta.cpp
| +-Raport.h / Raport.cpp
| +-ICinemaService.h      # Interfata abstracta
| +-Exceptii.h            # Ierarhia de exceptii
| +-Repository.h          # Template generic
| +-main_consola.cpp       # Entry-point aplicatie consola
| +-main_web.cpp           # Entry-point server HTTP C++
+-tests/
| +-test_cinema.cpp        # 17 teste unitare, framework propriu
+-docs/
| +-documentatie.md        # Acest document
| +-documentatie.pdf       # Versiune PDF
+-web/
| +-index.html             # Interfata web (single-page app)
| +-data.js                # Date initiale
+-date/                    # Fisiere de persistenta (pipe-delimited)
| +-filme.txt
| +-sali.txt
| +-spectacole.txt
| +-rezervari.txt
| +-angajati.txt
+-Makefile
+-CMakeLists.txt
+-README.md

```

2. Diagrama UML — clase si relatii

Diagrama completa (format Mermaid) se gaseste in documentatie.md.

Mai jos sunt relatiile principale:

Relatie	Tip	Descriere
Cinematograf -> Film	Agregare 1..*	Cinematograful detine filmele
Cinematograf -> Sala	Agregare 1..*	Cinematograful detine salile
Cinematograf -> Spectacol	Compozitie	Spectacolele nu exista fara cinema
Cinematograf -> Rezervare	Compozitie	Rezervarile apartin cinematografului
Spectacol -> Film	Asociere	Un spectacol are un film
Spectacol -> Sala	Asociere	Un spectacol are o sala
Rezervare -> Spectacol	Asociere	Rezervarea e pentru un spectacol
Rezervare -> Persoana	Asociere	Rezervarea apartine unui client
SalaVIP : Sala	Mostenire	VIP extinde sala normala
RezervareOnline : Rezervare	Mostenire	Online extinde rezervarea clasica

Ierarhii de mostenire:

```

ICinemaService <-- Cinematograf
Sala <-- SalaVIP
Rezervare <-- RezervareOnline
Persoana <-- Adult, Student, Elev, Pensionar, Copil
Angajat <-- Manager, Casier, Administrator
CinemaException <-- LocOcupatException, LocInvalidException, ...

```

3. Descrierea claselor

Film

Reprezinta un film disponibil in cinema.

Atribut	Tip	Descriere
Titlu	string	Titlul filmului
Categorie	string	Gen: Sci-Fi, Drama, Animatie...
Durata	int	Durata in minute
VarstaMinima	int	Restrictie de varsta (0 = toate varstele)
TipProiectie	string	2D sau 3D
Rating	double	Rating 1-10
Regizor, An, Limba	diverse	Metadate

Metode notabile: Este3D(), operator==, operator< (sortare alfabetica), operator<< pentru afisare in stream.

Sala

Sala fizica de cinema cu matrice dinamica de locuri.

Atribut	Tip	Descriere
NumarSala	string	Identificator sala (ex: A1, IMAX)
Capacitate	int	Total locuri
NumarRanduri, NumarColoane	int	Dimensiuni sala
Matrice	int**	Matrice alocata dinamic; 0=liber, 1=ocupat

Constructorul aloca matricea (AlocaMatrice()), destructorul o elibereaza (ElibereazaMatrice()). Constructorul de copiere si operator= fac **deep copy** (Rule of Three). Metode **virtual** pentru polimorfism: AfisareHarta(), EsteVIP(), GetSuplimentPret().

SalaVIP

Mostenire publica din Sala. Adauga facilitati premium.

Atribut suplimentar	Descriere
SuplimentPret	Multiplator pret (implicit 1.5 = +50%)
TipScaun	Recliner etc.
MeniuDisponibil	vector cu optiuni food and beverage

Suprascrie: AfisareHarta(), EsteVIP() returneaza true, GetSuplimentPret() returneaza 1.5.

Spectacol

O proiectie concreta: film + sala + data + ora.

Atribut	Tip	Descriere
film	Film*	Filmul proiectat
sala	Sala*	Sala in care ruleaza
data, ora	string	Format YYYY-MM-DD, HH:MM
pretBaza	double	Pretul de baza al biletului
ocupat	bool**	Matrice proprie (permite mai multe spectacole/sala)

RezervaLoc(r, c) arunca LocInvalidException sau LocOcupatException dupa caz.

LocuriOcupate() si ProcOcupare() furnizeaza statistici.

Persoana (abstracta)

Baza pentru toti clientii. Metode pure virtuale:

```
virtual float GetDiscount() const = 0; // 0.0 - 0.5
virtual string GetTip() const = 0; // "Adult", "Student"...
```

Subclase concrete si discount-urile lor:

Clasa	Discount
Adult	0%
Student	20%
Elev	15%
Pensionar	30%
Copil	50%

Rezervare

Bilet emis pentru un client la un spectacol, loc precis. Calculeaza automat

pretFinal prin CalculeazaPret(), aplicand toti factorii (sectiunea 8).

Stocheaza data emiterii. EsteOnline() returneaza false (suprascrisa in RezervareOnline).

RezervareOnline

Mostenire din Rezervare. Adauga:

- emailClient — adresa de email a cumparatorului
 - codConfirmare — generat automat: ONL-XXXX-id
 - EsteOnline() returneaza true
 - Suprascrie Afisare() pentru a include detaliile online
-

ICinemaService (interfata)

Contract abstract pentru orice implementare de cinema:

```
class ICinemaService {
public:
    virtual void      AdaugaFilm(Film*)                = 0;
    virtual Rezervare* FaRezervare(Persoana*, Spectacol*,
                                   int rand, int col)  = 0;
    virtual void      AfisareFilme() const            = 0;
    virtual ~ICinemaService() = default;
};
```

Permite scrierea de cod polimorfic si crearea de mock-uri pentru teste.

Repository (template)

Colectie generica parametrizata:

```
template<typename T>
class Repository {
    vector<T*> elemente;
    bool detineMemoria;
public:
    void      Adauga(T*);
    T*        GasestePrimul(function<bool(const T*)> pred) const;
    vector<T*> Filtreaza(function<bool(const T*)> pred) const;
    void      Sorteaza(function<bool(const T*, const T*)> cmp);
};
```

Exemplu de utilizare cu lambda:

```
auto filme3D = repo.Filtreaza([](const Film* f) {
    return f->Este3D();
});
```

SistemAutentificare

Gestioneaza sesiunile de login pentru staff. Verifica credentialele din date/angajati.txt (format: id|rol|nume|username|parola|salariu).

Roluri disponibile: Admin, Manager, Casier.

Metode principale: Autentifica(), VerificaDrepturiAdmin(), VerificaDrepturiManager().

Persistenta

Metode statice pentru salvare/incarcare in fisiere .txt cu separatorul |.

Asigura reluarea starii intre rulari ale aplicatiei.

Raport

Calculeaza statistici: top filme dupa numar de rezervari, incasari per sala, per categorii de film, per zi a saptamanii.

4. Concepte POO aplicate

Incapsulare

Toate attributele sunt private sau protected. Acces exclusiv prin getteri/setteri.

Constructorii valideaza datele si arunca exceptii la valori invalide (DateInvalideException).

Mostenire

Sase ierarhii de mostenire (vezi sectiunea 2).

Polimorfism

Metode virtuale apelate prin pointer la clasa de baza:

```
Sala* s = new SalaVIP("VIP1", 20, 4, 5);
s->AfisareHarta();           // => SalaVIP::AfisareHarta()
s->GetSuplimentPret();       // => 1.5 (nu 1.0)

ICinemaService* svc = new Cinematograf(...);
svc->AfisareFilme();         // => Cinematograf::AfisareFilme()
```

Templates (sabioane)

Repository este complet generic: Repository de Film, Sala, Rezervare etc.

Predicatele si comparatoarele sunt std::function, acceptand lambda-uri la apel.

Exceptii

Ierarhie proprie derivata din `std::exception`. Fiecare exceptie are mesaj precompilat si getteri pentru parametri (ex: `LocOcupatException::GetRand()`). Prinse selectiv sau generic cu `catch(const CinemaException& e)`.

Operator overloading

- `operator<<` pentru afisare in stream (toate clasele majore)
- `operator==` si `operator<` pentru Film si Spectacol (sortare)
- `operator=` cu garda de auto-asignare pentru clase cu memorie dinamica

Rule of Three

Clasele Sala si Spectacol detin memorie dinamica (int, **bool**) si implementeaza: constructor de copiere (deep copy), `operator=` (elibereaza memoria veche, deep copy), destructor (elibereaza memoria).

5. Interfata web si serverul HTTP C++

Arhitectura serverului

Serverul este implementat in `main_web.cpp` folosind POSIX sockets (socket, bind, listen, accept) fara nicio librarie externa. Fiecare conexiune HTTP este tratata intr-un thread separat (`std::thread`), cu un mutex global pentru protejarea datelor partajate.

Fluxul unui request:

1. `accept()` primeste conexiunea TCP
2. Se citeste linia de request (metoda + path + headers)
3. Se citeste body-ul (daca exista Content-Length)
4. Se dispatcheaza la handlerul corespunzator (GET /, POST /api/login, etc.)
5. Se returneaza raspunsul JSON sau fisierul static

Sesiuni (autentificare web)

Sesiunile sunt gestionate **in memorie** (nu in cookie-uri sau localStorage), printr-un map protejat de mutex:

```
struct Sesiune { string username; string rol; string nume; };
static map<string, Sesiune> sesiuni;    // token -> sesiune
static mutex sesiuniMutex;
```

La login se genereaza un token aleator (`TOKEN_random`), returnat in JSON.

Clientul il trimite la fiecare request prin header-ul X-Token.

La logout, sesiunea este stearsa din map.

Interfata web (web/index.html)

Single-page application in HTML/CSS/JavaScript pur, fara framework extern.

Functionalitate	Descriere
Login staff	Overlay de autentificare la deschidere
Vizitator	Acces fara cont pentru rezervari simple
Harta interactiva	Locuri colorate: liber/VIP/selectat/ocupat, live de la server
Mod consecutiv	Selectie automata N locuri consecutive pe un rand
Formular per bilet	Nume/varsta/tip/email individual pentru fiecare bilet selectat
Panou Admin	Adauga film, adauga spectacol, sterge rezervare
Incasari live	Statistici online/casa cu buton de stergere per rand
Actualizare automata	Contorul de bilete se actualizeaza dupa fiecare rezervare si la 30s

6. API REST — endpoint-uri

Toate raspunsurile sunt in format JSON.

Endpoint-urile protejate cer header-ul X-Token.

Metoda	Path	Auth	Descriere
POST	/api/login	Nu	Login; returneaza token, rol, nume
GET	/api/logout	Nu	Invalideaza sesiunea (token in query)
POST	/api/rezervare	Optional	Creeaza rezervare; RezervareOnline daca are email
GET	/api/locuri-ocupate	Nu	Locuri ocupate pentru spectacolID=X
GET	/api/rezervari	Da	Lista tuturor rezervarilor
DELETE	/api/rezervare	Da	Sterge rezervarea cu id=X
POST	/api/film	Da Admin/Manager	Adauga film nou
POST	/api/spectacol	Da Admin/Manager	Adauga spectacol nou
GET	/api/filme-toate	Da	Lista filme pentru selectul din admin
GET	/api/sali-toate	Da	Lista sali pentru selectul din admin
GET	/	Nu	Serveste web/index.html

Metoda	Path	Auth	Descriere
GET	/data.js	Nu	Serveste web/data.js

7. Fluxul unei rezervari

Console

Utilizatorul apeleaza `FaRezervare(client, spectacol, rand, col)` pe Cinematograf.
 Acesta delega catre `Spectacol::RezervaLoc(rand, col)` care arunca
`LocInvalidException` daca indicii sunt in afara salii, sau `LocOcupatException`
 daca locul este deja rezervat. Daca locul este liber, se creeaza obiectul
`Rezervare` care calculeaza `pretFinal` prin `CalculeazaPret()`. Daca varsta
 clientului este sub `VarstaMinima` a filmului, se arunca `VarstaInsuficientaException`
 si locul este eliberat.

Web

1. Utilizatorul deschide `http://localhost:8080`
2. Login staff SAU continua ca vizitator (fara cont)
3. Click pe un film -> selecteaza spectacol
4. Interfata face `GET /api/locuri-ocupate` -> harta live
5. Click pe locuri (mod normal sau mod consecutiv)
 Mod consecutiv: introduce N, activeaza modul,
 click pe primul loc -> N locuri selectate automat pe acelasi rand
6. Buton Rezerva N bilete -> formular per bilet
 (nume, varsta, tip, email - individual pentru fiecare bilet)
7. Confirmare -> `POST /api/rezervare` pentru fiecare loc
 Server apeleaza `Cinematograf::FaRezervare` (acelasi flux ca la consola)
8. Harta se actualizeaza live; contorul din hero se updateaza

8. Formula de calcul a pretului

```

pretFinal = pretBaza
    x (1 - discount_client)
    x multiplicator_zi
    x (loc_vip ? 1.30 : 1.00)
    x (sala_vip ? 1.50 : 1.00)
    x (film_3D ? 1.20 : 1.00)
  
```

Factor	Valori
discount_client	Adult 0% · Student 20% · Elev 15% · Pensionar 30% · Copil 50%
multiplicator_zi	Vineri 0.50 (-50%) · Sambata/Duminica 1.10 (+10%) · restul 1.00

Factor	Valori
loc_vip	Ultimele 2 randuri intr-o sala normala (+30%)
sala_vip	Intreaga sala de tip VIP (+50%)
film_3D	Tip proiectie 3D (+20%)

Ziua saptamanii se calculeaza din campul data al spectacolului (format YYYY-MM-DD) folosind algoritmul Tomohiko Sakamoto, deterministic si testabil.

Exemple de calcul

Pret baza	Client	Film	Zi	Sala	Pret final
30 RON	Adult	2D	Marti	Normala	30.00 RON
30 RON	Student	2D	Marti	Normala	24.00 RON
30 RON	Adult	3D	Marti	Normala	36.00 RON
30 RON	Adult	2D	Vineri	Normala	15.00 RON
30 RON	Adult	2D	Sambata	Normala	33.00 RON
30 RON	Adult	2D	Marti	VIP	45.00 RON
55 RON	Student	3D	Vineri	VIP	29.70 RON

9. Ierarhia exceptiilor

```
std::exception
+-- CinemaException          (mesaj general, what() override)
+-- LocOcupatException       (rand, coloana)
+-- LocInvalidException      (rand, coloana, maxRanduri, maxColoane)
+-- VarstaInsuficientaException (varstaClient, varstaMinima)
+-- DateInvalidException     (descriere problema)
+-- AutentificareException   (username/parola gresite)
+-- DrepturiInsuficienteException (rol fara permisiunea ceruta)
+-- ElementInexistentException (tip entitate + id)
+-- FisierException          (numeFisier, detalii eroare I/O)
```

Exceptiile de la nivelul nucleului C++ sunt traduse de serverul web in raspunsuri JSON de eroare: {"success":false,"error":"mesaj"}.

10. Strategie de testare

Fisierul tests/test_cinema.cpp contine un **mini-framework propriu** de testare

(macro-uri ASSERT_TRUE, ASSERT_EQ, ASSERT_NEAR, ASSERT_THROWS) fara dependinte externe.

Suite de teste (17 teste, 128 assertii)

Nr	Funcție test	Ce verifică
1	test_Film_Creare	Constructor și getteri Film
2	test_Sala_AlocareSiMatrice	Alocare matrice, ocupare loc, verificare
3	test_SalaVIP_Polimorfism	EsteVIP(), GetSuplimentPret() prin pointer Sala*
4	test_Repository_Template	Adauga, filtrare cu lambda, cautare
5	test_Rezervare_Valida	Rezervare cu succes; locul devine ocupat
6	test_Rezervare_LocOcupatException	Al doilea client pe același loc
7	test_Rezervare_LocInvalidException	Rand/coloana în afara salii
8	test_Rezervare_VarstaInsuficientaException	Minor la film 13+; locul rămâne liber
9	test_Pret_2D_vs_3D	Film 3D costa 1.20x mai mult
10	test_Pret_DiscountStudent	Student plătește 80% din pret
11	test_Pret_Vineri_50pct	Vineri pretul este înjumătățit
12	test_Pret_Weekend_Supliment	Sambata/Duminica +10%
13	test_Pret_SalaVIP_Supliment	Sala VIP +50%
14	test_RezervareOnline_Cod	Email stocat, cod confirmare generat
15	test_Cinematograf_Cautari	GasesteFilmDupaID, GasesteSalaDupaID
16	test_ICinemaService_Polimorfism	Pointer ICinemaService* -> Cinematograf
17	test_Spectacol_Statistici	LocuriOcupate(), ProcOcupare()

Rulare teste

```
cd cinema-poo
make test
```

11. Instrucțiuni de build și rulare

Cerinte sistem

- Linux sau WSL (Ubuntu 22.04+)
- Compilator: sudo apt install g++ make

Aplicatie consola

```
cd cinema-poo
make          # compileaza executabilul ./cinema
./cinema      # lanseaza aplicatia consola interactiva
```

Interfata web (server HTTP C++)

```
make web          # compileaza ./cinema_web (necesita g++ + pthread)
./cinema_web      # porneste serverul pe portul 8080
```

Deschide in browser: `http://localhost:8080`

Credentiale demo: admin/admin, manager/manager, casier/casier

Teste unitare

```
make test      # compileaza si ruleaza cele 17 teste unitare
```

Curatare fisiere compilate

```
make clean
```

12. Posibile imbunatatiri

1. **Smart pointers** — Inlocuirea new/delete cu `std::unique_ptr` sau `std::shared_ptr` pentru eliminarea riscului de memory leak la exceptii.

2. **Baza de date relationala** — Inlocuirea fisierelor .txt cu SQLite pentru tranzactii atomice, interogari complexe si integritate referentiala.

3. **Concurenta** — Mutexuri per-spectacol in locul mutexului global, permitand rezervari simultane in sali diferite fara blocare totala.

4. **Autentificare JWT** — Tokenurile actuale sunt siruri aleatoare simple. JSON Web Token ar adauga expirare si semnatura criptografica.

5. **WebSocket** — Actualizarea hartii salii in timp real pentru toti utilizatorii conectati simultan, in loc de polling periodic.

6. **Test coverage** — Extinderea testelor la Persistenta, Raport si SistemAutentificare; integrare gcov pentru masurarea acoperirii.

7. **Internationalizare** — Fisier de mesaje .po pentru traducere usoara in alte limbi.

8. **CI/CD** — Integrare GitHub Actions pentru build + teste automate la fiecare commit (verifica ca make test trece pe Ubuntu).

*Document realizat ca parte a proiectului de Programare Orientata pe Obiecte,
Tema 3121B. Codul sursa complet se gaseste in repository-ul GitHub al cursului.*